

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №13

Тема: Сортування методом обміну (бульбашкове сортування)

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Детально опанувати принцип роботи бульбашкового сортування (сортування методом обміну), навчитися застосовувати оптимізацію з прапорцем раннього завершення для перетворення алгоритму на адаптивний, дослідити на практиці кількість проходів і порівнянь для різних типів вхідних даних.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Принцип роботи. Алгоритм виконує серію проходів по масиву; на кожному проході послідовно порівнюються сусідні пари елементів ($a[j]$, $a[j+1]$); якщо елементи розташовані в неправильному порядку ($a[j] > a[j+1]$ для сортування за зростанням), вони міняються місцями. Назва "бульбашкове" пов'язана з тим, що на кожному проході найбільший ще не відсортований елемент поступово "спливає" (як бульбашка) до свого правильного місця в кінці масиву.

2. Базова версія. Виконує рівно $n-1$ проходів, на кожному i -му проході ($i=0, \dots, n-2$) порівнюються $n-1-i$ пар сусідніх елементів (оскільки останні i елементів уже гарантовано відсортовані попередніми проходами). Загальна кількість порівнянь: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$, незалежно від того, чи був масив уже відсортований — головний недолік базової версії.

3. Оптимізація прапорцем раннього завершення. Якщо під час чергового проходу не відбулося жодного обміну — це означає, що масив уже повністю відсортований, і подальші проходи не потрібні. Оптимізована версія заводить булеву змінну `swapped`, яка встановлюється в `True` при будь-якому обміні на поточному проході; якщо після завершення проходу `swapped` залишилася `False` — цикл переривається. Це перетворює алгоритм на адаптивний: для вже відсортованого масиву достатньо лише одного проходу ($n-1$ порівнянь, 0 обмінів) – складність $O(n)$ у найкращому випадку.

4. Різновид — шейкерне (коктейльне) сортування (Cocktail Shaker Sort). Модифікація, при якій проходи виконуються по чергові в обидва боки (зліва направо, потім справа наліво), що дозволяє за один подвійний прохід "підняти" на місце як найбільший, так і найменший ще не відсортовані елементи; корисно для масивів з "черепашками" — малими елементами в кінці масиву, які базове бульбашкове сортування пересуває лише на одну позицію за прохід.

5. Складність:

Випадок	Базова версія	Оптимізована версія
Найкращий (відсортований масив)	$O(n^2)$	$O(n)$
Середній	$O(n^2)$	$O(n^2)$
Найгірший (зворотний порядок)	$O(n^2)$	$O(n^2)$

Хід роботи

Завдання 1. Повна ручна трасировка бульбашкового сортування

Дано масив: [5, 1, 4, 2, 8, 0, 3]. Виконати вручну повне бульбашкове сортування (усі проходи до повного впорядкування), фіксуючи стан масиву та кількість обмінів після кожного проходу.

Порядок виконання:

1. Пронумеруйте проходи від 1.
2. На проході 1 порівняйте пари $(a[0], a[1])$, $(a[1], a[2])$, ..., $(a[5], a[6])$ послідовно, міняючи місцями елементи за потреби, і записуйте стан масиву після кожного обміну (або принаймні після кожної пари порівнянь).
3. Зафіксуйте стан масиву й кількість обмінів після завершення проходу 1.
4. Повторіть аналогічно для проходу 2 (порівнюючи вже на одну пару менше, оскільки останній елемент гарантовано на місці), і так до останнього проходу.
5. Складіть підсумкову таблицю "№ проходу — стан масиву після проходу — кількість порівнянь — кількість обмінів".
6. Підрахуйте загальну кількість порівнянь і обмінів за весь процес сортування та порівняйте з теоретичною формулою $n(n-1)/2$ для $n=7$.

Завдання 2. Трасировка оптимізованої версії з раннім завершенням

Дано частково відсортований масив: [1, 2, 3, 5, 4, 6, 7]. Виконати вручну оптимізоване бульбашкове сортування (з прапорцем `swapped`) і показати, на якому проході алгоритм завершиться достроково.

Порядок виконання:

1. Виконайте прохід 1: порівняйте всі сусідні пари, зафіксувавши, чи стався хоча б один обмін (у цьому масиві обмін станеться лише один раз — між 5 і 4).
2. Запишіть стан масиву після проходу 1 та значення прапорця `swapped` (True, оскільки був один обмін).
3. Виконайте прохід 2: порівняйте всі сусідні пари в уже майже відсортованому масиві; якщо жодного обміну не відбулося — встановіть `swapped = False`.
4. Оскільки `swapped = False` після проходу 2, алгоритм завершується, не виконуючи подальших проходів.
5. Порахуйте загальну кількість порівнянь за 2 проходи та порівняйте з тим, скільки порівнянь виконала б базова версія без оптимізації (усі $n-1=6$ проходів).
6. Зробіть висновок про економію обчислень завдяки оптимізації для цього практично майже відсортованого масиву.

Завдання 3. Програмна реалізація базової та оптимізованої версій

Реалізувати мовою Python базове бульбашкове сортування та його оптимізовану версію з раннім завершенням, перевірити коректність обох.

Порядок виконання:

1. Напишіть функцію `bubble_sort_basic(arr)`, що виконує повний подвійний цикл (зовнішній `for i in range(n-1)`, внутрішній `for j in range(n-1-i)`) без будь-яких оптимізацій, рахуючи порівняння, обміни й кількість виконаних проходів.
2. Напишіть функцію `bubble_sort_optimized(arr)` за тією ж схемою, але з булевою змінною `swapped`, яка скидається на `False` на початку кожного проходу і встановлюється в `True` при обміні; після внутрішнього циклу перевірте `if not swapped: break`.
3. Перевірте обидві функції на тестовому масиві, порівнявши результат з `sorted()`.
4. Порівняйте кількість пройдених ітерацій зовнішнього циклу (проходів) для обох версій на одному й тому самому вхідному масиві.

Приклад реалізації мовою Python:

```
python
def bubble_sort_basic(arr):
    a = arr.copy(); n = len(a); c = s = 0; passes = 0
    for i in range(n-1):
        passes += 1
        for j in range(n-1-i):
            c += 1
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]; s += 1
    return a, c, s, passes

def bubble_sort_optimized(arr):
    a = arr.copy(); n = len(a); c = s = 0; passes = 0
    for i in range(n-1):
        passes += 1
        swapped = False
        for j in range(n-1-i):
            c += 1
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]; s += 1
                swapped = True
        if not swapped:
            break
    return a, c, s, passes

test = [5, 2, 9, 1, 5, 6, 3]
assert bubble_sort_basic(test)[0] == bubble_sort_optimized(test)[0] == sorted(test)
print("Коректність підтверджено:", sorted(test))
```

Результат виконання програми:

Коректність підтверджено: [1, 2, 3, 5, 5, 6, 9]

Завдання 4. Порівняльне дослідження на різних профілях даних

Порівняти кількість порівнянь і проходів базової та оптимізованої версій бульбашкового сортування на трьох профілях вхідних даних: випадковий, відсортований, зворотно відсортований масив.

Порядок виконання:

1. Для фіксованого розміру n (наприклад, 1000) згенеруйте три масиви: випадковий, вже відсортований, відсортований у зворотному порядку.
2. Виконайте на кожному масиві обидві версії сортування (`bubble_sort_basic` і `bubble_sort_optimized`), зафіксувавши кількість порівнянь і кількість пройдених проходів для кожної комбінації.
3. Занесіть результати в таблицю: "профіль даних — базова(порівнянь) — базова(проходів) — оптимізована(порівнянь) — оптимізована(проходів)".
4. Особливу увагу зверніть на випадок "відсортований масив" — порівняйте кількість проходів і порівнянь оптимізованої версії з базовою.
5. Побудуйте (за наявності часу) графік залежності кількості порівнянь від розміру масиву для обох версій на відсортованих даних, щоб наочно показати перехід від $O(n^2)$ до $O(n)$.
6. Сформулюйте висновок про практичну користь оптимізації раннього завершення.

Приклад реалізації мовою Python:

```
python
import random

profiles = {
    "випадковий": lambda n: [random.randint(0, 10000) for _ in range(n)],
    "відсортований": lambda n: list(range(n)),
    "зворотно відсортований": lambda n: list(range(n, 0, -1)),
}

n = 1000
print(f"{'профіль':25s} {'базове(пор.):>13} {'базове(прох.):>14} {'оптим.(пор.):>13} {'оптим.(прох.):>14}")
for name, gen in profiles.items():
    arr = gen(n)
    _, c1, _, p1 = bubble_sort_basic(arr)
    _, c2, _, p2 = bubble_sort_optimized(arr)
    print(f"{name:25s} {c1:>13} {p1:>14} {c2:>13} {p2:>14}")
```

Результат виконання програми:

профіль	базове(пор.)	базове(прох.)	оптим.(пор.)	оптим.(прох.)
випадковий	499500	999	499094	971
відсортований	499500	999	999	1
зворотно відсортований	499500	999	499500	999

Висновок за результатами: базова версія виконує рівно 499500 порівнянь і 999 проходів **незалежно від профілю даних** — підтверджуючи її неадаптивність. Оптимізована версія на випадковому й зворотно відсортованому масивах працює майже так само, як і базова (оскільки обміни трапляються практично на кожному проході аж до кінця). Але на вже **відсортованому** масиві оптимізована версія завершується всього за **1 прохід і 999 порівнянь** замість 999 проходів і 499500 порівнянь у базовій версії — прискорення у 500 разів. Це наочно демонструє практичну цінність, здавалося б, простої оптимізації: без зміни асимптотики в середньому й найгіршому випадках вона перетворює найкращий випадок з $O(n^2)$ на $O(n)$, що особливо важливо для систем, де дані часто надходять уже (майже) відсортованими.

Контрольні питання

1. Опишіть принцип роботи бульбашкового сортування. Чому воно отримало таку назву?
2. Скільки порівнянь виконує базове бульбашкове сортування для масиву розміром n ? Виведіть формулу.
3. Яку проблему базової версії вирішує оптимізація з прапорцем раннього завершення?
4. Яка складність оптимізованої версії бульбашкового сортування в найкращому випадку? За яких вхідних даних вона досягається?
5. Чи змінюється складність оптимізованого бульбашкового сортування в найгіршому й середньому випадках порівняно з базовою версією?
6. У чому суть шейкерного (коктейльного) сортування і яку проблему базового бульбашкового сортування воно покращує?
7. Чи є бульбашкове сортування стійким? Обґрунтуйте відповідь.

Зміст звіту

1. Тема, мета роботи.
2. Розгорнуті ручні трасировки (таблиці) до завдань 1 і 2.
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Порівняльна таблиця й висновки про ефективність оптимізації.
5. Висновки.